

The ULTIMA 2.0 project is focused around the design and implementation of a multitasking operating system. The goal of the ULTIMA project would be to progressively build essential components from an operating system across multiple phases, creating an environment capable of managing tasks, resources, and system-level operations. With each given phase, new subsystems are added to add onto the overall complexity and functionality of the system. This will provide the end goal of simulating an operating system that manages multiple tasks, coordinates access to shared resources, allocates CPU time, and providing an understanding of multitasking and concurrency.

Phase 1, the foundation is set through the implementation of the scheduler and utilizing semaphores. The focus is on building a scheduler that maintains a collection of tasks and cycles through them using a time-based approach. With each task, it is tracked on its state and execution timing, allowing the system to simulate transitions such as ready, running, and blocked. This sets the base for understanding how the OS distributes CPU time and handles multiple processes.

Phase 2 expands the functionality of ULTIMA by introducing inter-process communication or IPC through a message passing system. This phase allows tasks within the system to communicate by sending and receiving structured messages through dedicated mailboxes associated with each task. The IPC system introduces support for message queues, mailbox management, and synchronized access to shared communication structures. By integrating messages passing into the scheduler-based environment introduced in Phase 1, the system now simulates how modern operating systems allow processes to coordinate and exchange information safely during execution. This phase further enhances the realism of the operating system simulation by demonstrating communication and coordination between concurrent tasks.

Phase 3 further expands the functionality of ULTIMA by introducing a Memory Management Unit or MMU. This phase allows tasks to request, use, read, write, and free memory within a fixed-size memory space. Memory is allocated using a block-based, first fit strategy, and each task is assigned to a unique memory handle stored in its control block to track ownership. The system maintains a linked list of memory segments to manage free and used space, while also providing memory dumps and usage tables to visualize the state of the memory. By integrating memory management with the scheduler and IPC systems from previous phase, this phase enhances the realism of simulator by modeling how modern operating systems manage memory for multiple concurrent tasks.